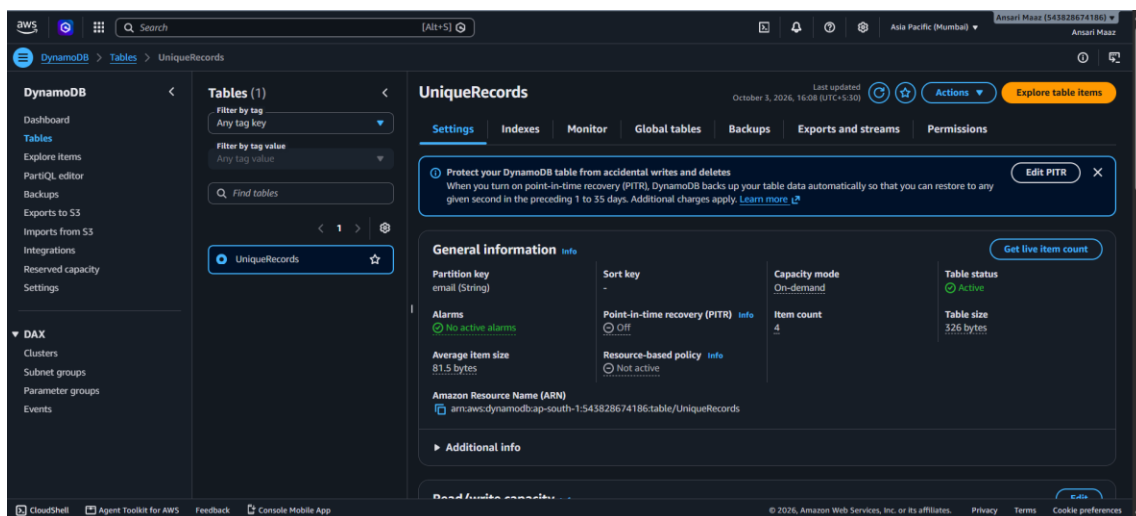


Data Redundancy Removal System

- > Identity Duplicate Records
- > validate incoming Traffic
- > prevent Duplicate database entries.
- > improve database consistency
- > monitor application activity

1) Create DynamoDB Table (name > UniqueRecords)

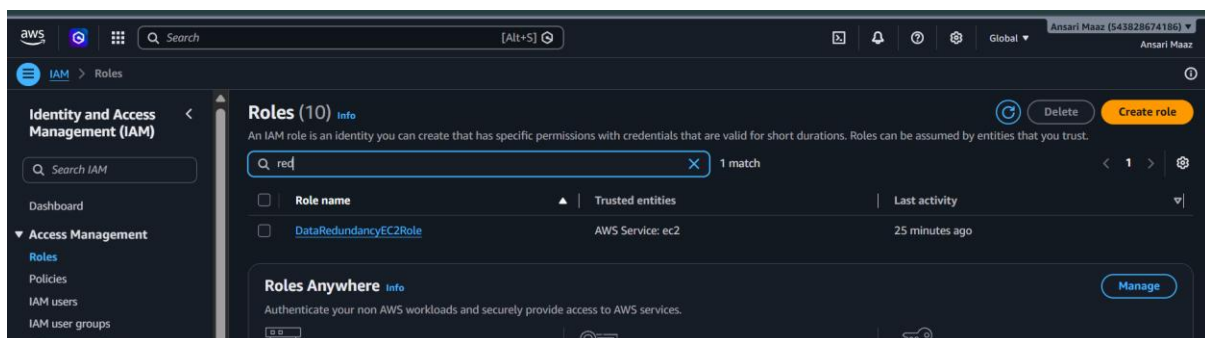
> partition key == email



2) Create IAM role

> ec2 full access

Name === DataRedundancyEC2Role

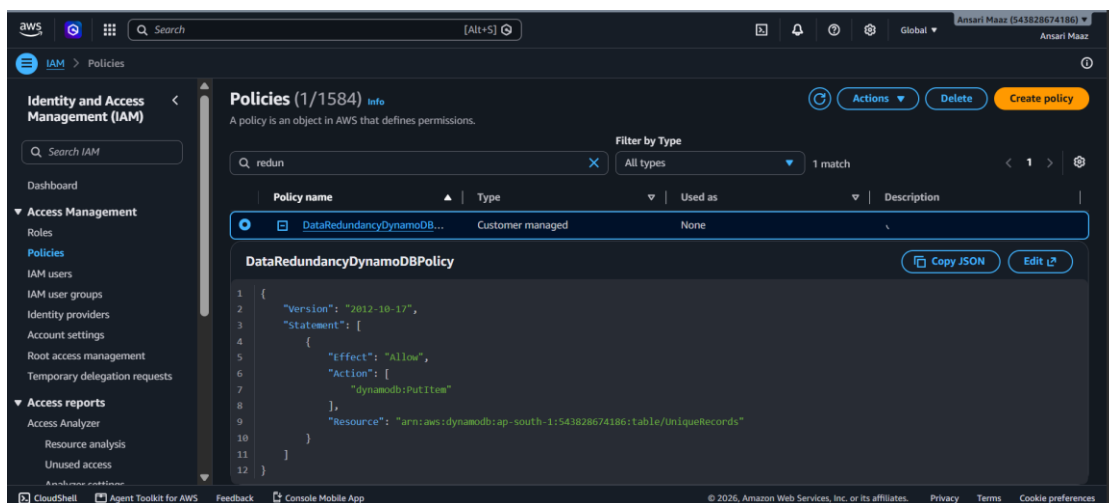


2.2) policy

> Open IAM → Policies → Create policy → JSON

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Action": [  
        "dynamodb:PutItem"  
      ],  
      "Resource": "arn:aws:dynamodb:ap-south-1:YOUR_ACCOUNT_ID:table/UniqueRecords"  
    }  
  ]  
}
```

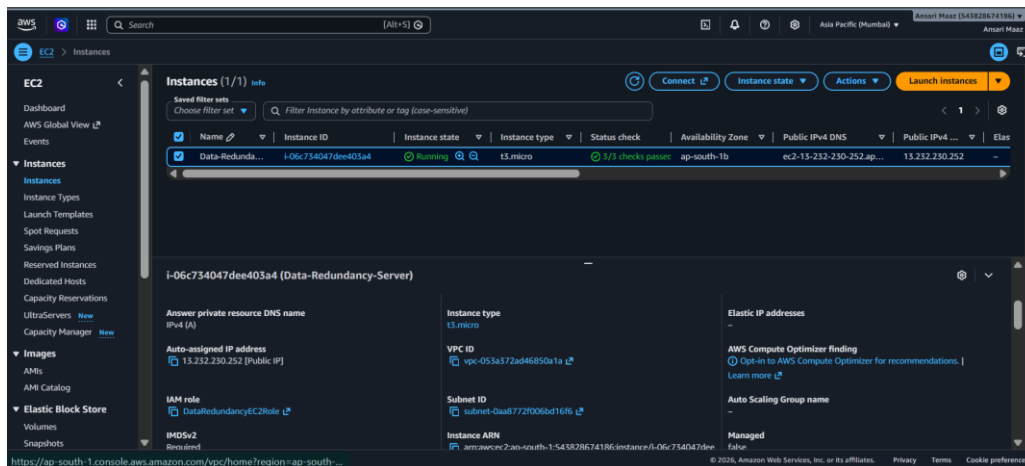
(name === DataRedundancyDynamoDBPolicy)



3) EC2 instance

> Under **Advanced details**, select the IAM instance profile associated with DataRedundancyEC2Role.

> launch ec2 instance



4) connect instance (ssh)

Open terminal / cmd

```
>python3 --version
>sudo dnf install python3 python3-pip -y
>mkdir data-redundancy-project
>cd data-redundancy-project
>touch app.py
>nano app.py
```

[python code..]

import logging

import re

from datetime import datetime, timezone

import boto3

from botocore.exceptions import ClientError

Configure application logging

logging.basicConfig(

level=logging.INFO,

```
    format="%asctime)s - %(levelname)s - %(message)s"
)
```

```
logger = logging.getLogger(__name__)
```

```
# AWS configuration
```

```
REGION = "ap-south-1"
```

```
TABLE_NAME = "UniqueRecords"
```

```
# Connect using the EC2 IAM role
```

```
dynamodb = boto3.resource(
```

```
    "dynamodb",
```

```
    region_name=REGION
```

```
)
```

```
table = dynamodb.Table(TABLE_NAME)
```

```
def validate_record(name, email, city):
```

```
    """Validate and normalize the incoming record."""
```

```
    if not all(
```

```
        isinstance(value, str)
```

```
        for value in (name, email, city)
```

```
):
```

```
        return None, "All fields must be text"
```

```
name = name.strip()
```

```
email = email.strip().lower()
```

```
city = city.strip()
```

```
if not name or not email or not city:
```

```
    return None, "All fields are required"
```

```
# Basic email-format validation
```

```
if not re.fullmatch(
```

```
    r"^[^\s]+@[^\s]+\.[^\s]+",
```

```
    email
```

```
):
```

```
    return None, "Invalid email format"
```

```
return {
```

```
    "name": name,
```

```
    "email": email,
```

```
    "city": city
```

```
}, None
```

```
def add_record(name, email, city):
```

```
    """Insert a record only if its email does not exist."""
```

```
record, validation_error = validate_record(
```

```
    name,
```

```
    email,
```

```
    city
```

```
)
```

```
if validation_error:
    logger.warning(
        "Record rejected: validation failed"
    )

    return {
        "status": "INVALID",
        "message": validation_error
    }

record["created_at"] = (
    datetime.now(timezone.utc).isoformat()
)

try:
    # Atomic conditional write
    table.put_item(
        Item=record,
        ConditionExpression=(
            "attribute_not_exists(email)"
        )
    )

    logger.info("Unique record inserted")

    return {
        "status": "INSERTED",
        "message": "Record stored successfully"
```

```
}
```

```
except ClientError as error:
```

```
    error_code = error.response.get(
        "Error", {}
    ).get("Code")
```

```
    if error_code == "ConditionalCheckFailedException":
```

```
        logger.info("Duplicate record rejected")
```

```
    return {
        "status": "DUPLICATE",
        "message": "Record already exists"
    }
```

```
    logger.exception("Database operation failed")
```

```
    return {
        "status": "ERROR",
        "message": "Database operation failed"
    }
```

```
if __name__ == "__main__":
```

```
    test_records = [
        ("Rahul", "rahul@gmail.com", "Mumbai"),
```

```
(
    ("Ayesha", "ayesha@gmail.com", "Pune"),
    ("Rahul", "rahul@gmail.com", "Mumbai"),
    ("Imran", "imran@gmail.com", "Hyderabad"),
    ("Ayesha", "ayesha@gmail.com", "Pune"),
    ("Invalid User", "wrong-email", "Mumbai")
]
```

for record in test_records:

```
    result = add_record(*record)
```

```
print(result)
```

[python code....]

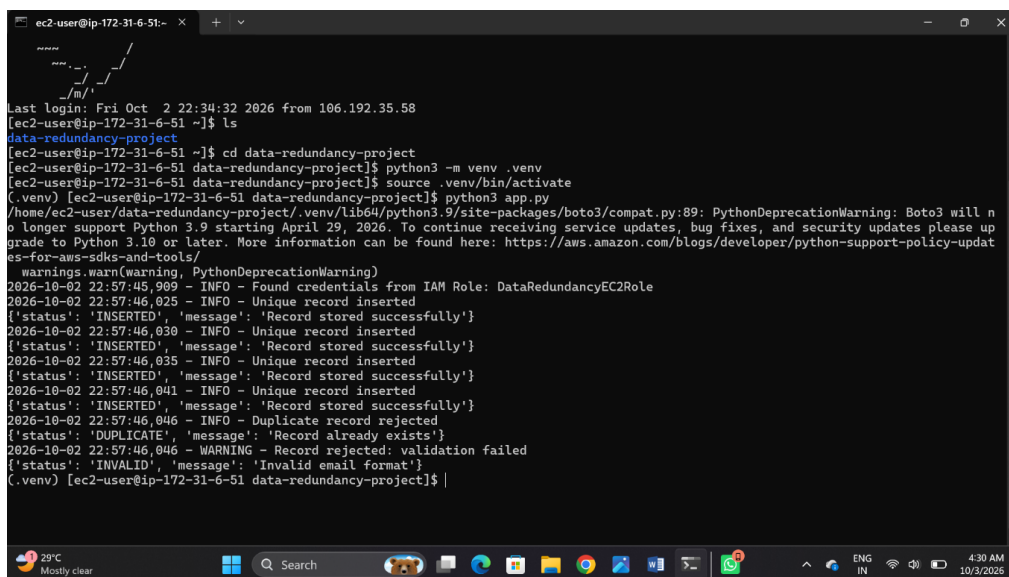
```
>python3 -m pip install boto3
```

```
>python3 -m venv .venv
```

```
>source .venv/bin/activate
```

```
>python -m pip install boto3
```

```
>python app.py
```



```
ec2-user@ip-172-31-6-51:~$ 
Last login: Fri Oct  2 22:34:32 2026 from 106.192.35.58
[ec2-user@ip-172-31-6-51 ~]$ ls
data-redundancy-project
[ec2-user@ip-172-31-6-51 ~]$ cd data-redundancy-project
[ec2-user@ip-172-31-6-51 data-redundancy-project]$ python3 -m venv .venv
[ec2-user@ip-172-31-6-51 data-redundancy-project]$ source .venv/bin/activate
(.venv) [ec2-user@ip-172-31-6-51 data-redundancy-project]$ python3 app.py
/home/ec2-user/data-redundancy-project/.venv/lib64/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting April 29, 2026. To continue receiving service updates, bug fixes, and security updates please upgrade to Python 3.10 or later. More information can be found here: https://aws.amazon.com/blogs/developer/python-support-policy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
2026-10-02 22:57:45,909 - INFO - Found credentials from IAM Role: DataRedundancyEC2Role
2026-10-02 22:57:46,025 - INFO - Unique record inserted
{'status': 'INSERTED', 'message': 'Record stored successfully'}
2026-10-02 22:57:46,030 - INFO - Unique record inserted
{'status': 'INSERTED', 'message': 'Record stored successfully'}
2026-10-02 22:57:46,035 - INFO - Unique record inserted
{'status': 'INSERTED', 'message': 'Record stored successfully'}
2026-10-02 22:57:46,041 - INFO - Unique record inserted
{'status': 'INSERTED', 'message': 'Record stored successfully'}
2026-10-02 22:57:46,046 - INFO - Duplicate record rejected
{'status': 'DUPLICATE', 'message': 'Record already exists'}
2026-10-02 22:57:46,046 - WARNING - Record rejected: validation failed
{'status': 'INVALID', 'message': 'Invalid email format'}
(.venv) [ec2-user@ip-172-31-6-51 data-redundancy-project]$
```


our Data :

```
test_records = [
    ("Rahul", "rahul@gmail.com", "Mumbai"),
    ("Ayesha", "ayesha@gmail.com", "Pune"),
    ("Rahul", "rhul@gmail.com", "Mumbai"),
    ("Imran", "imran@gmail.com", "Hyderabad"),
    ("Ayesha", "ayesha@gmail.com", "Pune"),
    ("Invalid User", "wrong-email", "Mumbai")
]
```

5) open DynamoDB and check table

The screenshot shows the AWS DynamoDB console interface. On the left, the 'DynamoDB' sidebar is visible with options like 'Dashboard', 'Tables', 'Explore Items', 'PartiQL editor', 'Backups', 'Exports to S3', 'Imports from S3', 'Integrations', 'Reserved capacity', and 'Settings'. The main panel shows the 'UniqueRecords' table. A 'Scan' operation has been performed, returning 4 items. The table structure is as follows:

email (String)	city	created_at	name
rahul@gmail.com	Mumbai	2026-10-02...	Rahul
ayesha@gmail.com	Pune	2026-10-02...	Ayesha
rahul@gmail.com	Mumbai	2026-10-02...	Rahul
imran@gmail.com	Hyderabad	2026-10-02...	Imran

Run again python file

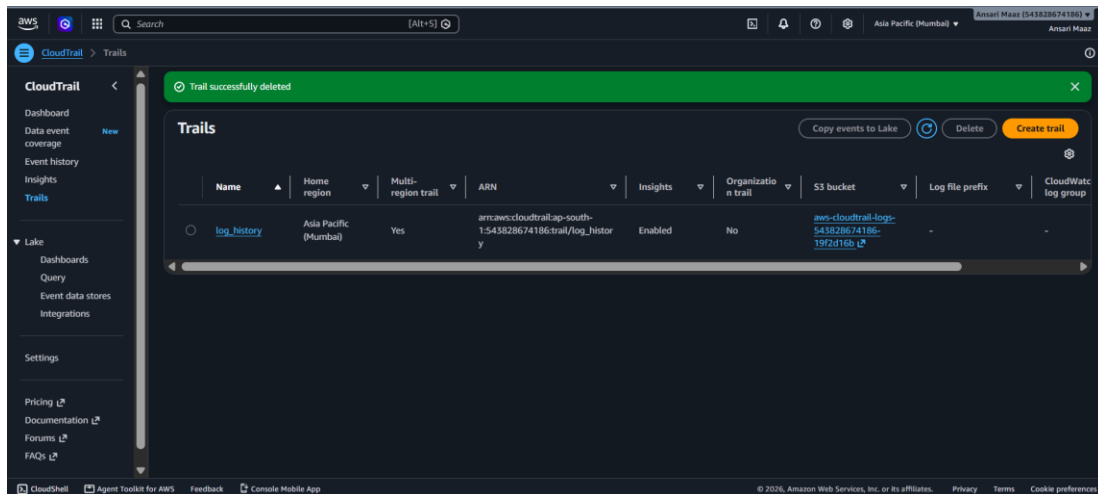
>python app.py

```
(.venv) [ec2-user@ip-172-31-6-51 data-redundancy-project]$ python3 app.py
/home/ec2-user/data-redundancy-project/.venv/Lib64/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting April 29, 2026. To continue receiving service updates, bug fixes, and security updates please upgrade to Python 3.10 or later. More information can be found here: https://aws.amazon.com/blogs/developer/python-support-policy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
2026-10-02 23:14:19,114 - INFO - Found credentials from IAM Role: DataRedundancyEC2Role
2026-10-02 23:14:19,228 - INFO - Duplicate record rejected
{'status': 'DUPLICATE', 'message': 'Record already exists'}
2026-10-02 23:14:19,232 - INFO - Duplicate record rejected
{'status': 'DUPLICATE', 'message': 'Record already exists'}
2026-10-02 23:14:19,236 - INFO - Duplicate record rejected
{'status': 'DUPLICATE', 'message': 'Record already exists'}
2026-10-02 23:14:19,241 - INFO - Duplicate record rejected
{'status': 'DUPLICATE', 'message': 'Record already exists'}
2026-10-02 23:14:19,245 - INFO - Duplicate record rejected
{'status': 'DUPLICATE', 'message': 'Record already exists'}
2026-10-02 23:14:19,245 - WARNING - Record rejected: validation failed
{'status': 'INVALID', 'message': 'Invalid email format'}
(.venv) [ec2-user@ip-172-31-6-51 data-redundancy-project]$
```

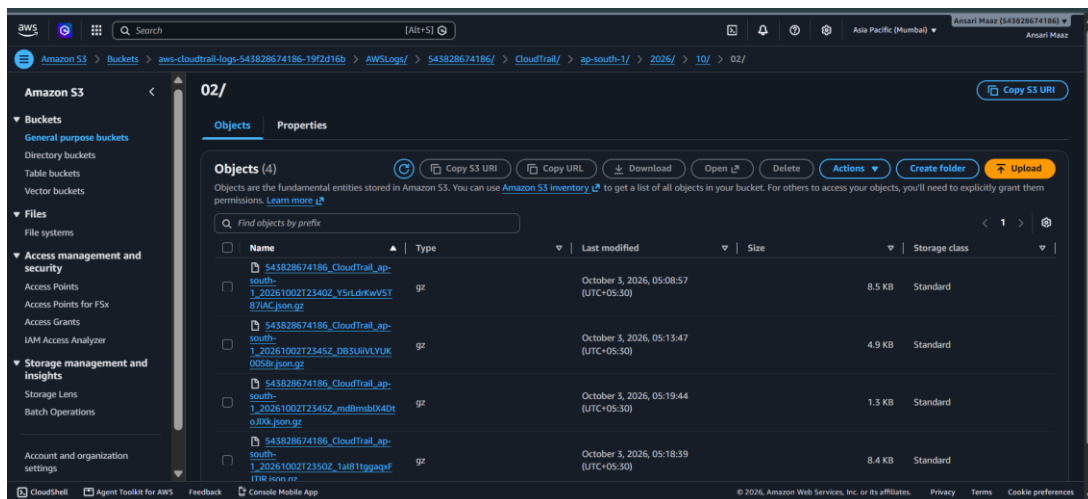
> cat app.log

6) Configure cloud trial(events and logs)

>with s3



>events and logs



This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Error>
  <Code>AccessDenied</Code>
  <Message>Access Denied</Message>
  <RequestId>RMCK2NX722JPT8EZ</RequestId>
  <HostId>5rD80VmdSTjTB1o3na1Zf4D1HKnzx5OUxE2WSMjERf58wOPdr2llydJ0aCLhLiRGHeYXehw5sTA8=</HostId>
</Error>
```